A Project Report on

Implementation of an Explainable Deep Learning Model for Transparent and Ethical Decision Support Systems

in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

Submitted by

M. Ratnakarun	22B81A1274
P. Harika	22B81A1293
Y. Kundana Kotesh	23B85A1212
Y. Meghana	22B81A12D0

Under the Esteemed Guidance of

Smt. T. Satya Nagamani

Assistant Professor, Department of IT

DEPARTMENT OF INFORMATION TECHNOLOGY

SIR CR REDDY COLLEGE OF ENGINEERING

Approved by AICTE & Accredited by NBA & NAAC

Affiliated to Jawaharlal Nehru Technological University, Kakinada

ELURU-534007

2025-26

SIR C R REDDY COLLEGE OF ENGINEERING

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that this project report entitled “Implementation of an Explainable Deep Learning Model for Transparent and Ethical Decision Support Systems” being Submitted by M. Ratnakarun(22B81A1274), P. Harika(22B81A1293), Y. Kundana Kotesh(23B5A1212), Y. Meghana(22B81A12D0) in partial fulfillment for the award of the Degree of Bachelor of Technology in Information Technology to the JNTU KAKINADA is a record of Bonafide work carried out under our guidance and supervision. The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree.

T. Satya Nagamani
PROJECT GUIDE
Department of IT
Sir CRR College of Engineering

Dr. K. Satyanarayana
HEAD OF THE DEPARTMENT
Department of IT
Sir CRR College of Engineering

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We wish to express our sincere thanks to various personalities who were responsible for the successful completion of this project.

We thank our principal, **Dr. K. VENKATESWARA RAO**, for providing the necessary infrastructure required for our project.

We are grateful to **Dr. K. SATYANARAYANA**, Head of the Information Technology department, for providing the necessary facilities for completing the project in specified time.

We express our deep-felt gratitude to Smt. **T. SATYA NAGAMANI**, as her valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time.

Our special thanks to librarian Smt. **D. LAKSHMI KUMARI**, and to the entire library staff Sir C.R.R College of Engineering, for providing the necessary library facilities.

We express our earnest thanks to faculty members and non-teaching staff of IT for extending their valuable support.

PROJECT MEMBERS

M. Ratnakarun	22B81A1274
P. Harika	22B81A1293
Y. Kundana Kotesh	23B85A1212
Y. Meghana	22B81A12D0

ABSTRACT

The project addresses the black-box nature of deep learning, which often lacks the transparency required for critical, high-stakes decisions. To solve this, we implemented an explainable deep learning framework that balances high predictive performance with robust interpretability. Our methodology integrates neural network architectures with post-hoc explanation techniques like SHAP and LIME to transform complex outputs into human-understandable justifications. Quantitatively, the system achieved competitive accuracy while qualitatively providing the rich explanatory insights necessary for ethical accountability. Ultimately, this work establishes transparency as a fundamental requirement for building trustworthy and responsible AI-driven decision support systems.

DECLARATION

We, students of IV/IV BTech II Semester, Department of Information Technology, SIR CRR COLLEGE OF ENGINEERING, has successfully completed the project work entitled "**IMPLEMENTATION OF AN EXPLAINABLE DEEP LEARNING MODEL FOR TRANSPARENT AND ETHICAL DECISION SUPPORT SYSTEMS**", which has been carried out by we under the esteemed guidance of Smt T. Satya Nagamani. I declare that the above work is not submitted at any university for the award of any degree.

M. Ratnakarun 22B81A1274

CONTENTS

LIST OF TABLES

LIST OF FIGURES

1. INTRODUCTION

In recent years, deep learning has emerged as a powerful technique for building intelligent decision support systems due to its ability to analyze complex data and produce highly accurate predictions. Such systems are increasingly used in critical domains like healthcare, finance, and risk assessment, where decisions can significantly impact individuals and organizations. However, traditional deep learning models often operate as black-box systems, making it difficult to understand how decisions are derived.

The lack of transparency in these models raises concerns related to trust, accountability, and ethical compliance. To overcome these challenges, Explainable Artificial Intelligence (XAI) has been introduced to provide insights into the decision-making process of deep learning models. By integrating explain ability techniques with deep learning, decision support systems can become more transparent, interpretable, and ethically aligned, enabling users to understand and trust the system's predictions.

1.1 Problem Statement

Although deep learning models provide high accuracy, their black-box nature limits their adoption in real-world applications that demand transparency and accountability. Stakeholders often find it difficult to interpret model predictions, making it challenging to justify decisions or identify potential biases in the system. This lack of explainability can lead to ethical issues, reduced user trust, and difficulty in regulatory compliance.

Therefore, there is a need for a decision support system that not only delivers accurate predictions but also provides clear and understandable explanations for its decisions. Addressing this problem requires the development of an explainable deep learning framework that balances model performance with interpretability and ethical responsibility.

1.2 Objective

The main objective of this project is to design and implement an explainable deep learning–based decision support system that ensures transparency and ethical decision-making. The specific objectives of the project are:

- To develop a deep learning model capable of making accurate decisions based on input data.
- To incorporate explainability techniques that reveal the contribution of input features to model predictions.
- To enhance user trust by providing transparent and interpretable decision outcomes.
- To evaluate the performance of the system using appropriate metrics while maintaining interpretability.

1.3 Scope of the Project

The scope of this project focuses on the development and evaluation of an explainable deep learning decision support system using structured data. The project covers data preprocessing, model training, performance evaluation, and implementation of explainability techniques to interpret model decisions. The system is designed to demonstrate transparency and ethical decision-making rather than domain-specific deployment.

While the project provides a foundational framework for explainable decision support systems, it can be extended in the future to include real-world datasets, advanced deep learning architectures, and more sophisticated XAI techniques. The proposed approach can be adapted to various application domains where transparent and trustworthy decision-making is required.

2. LITERATURE REVIEW

The rapid advancement of artificial intelligence in recent years has significantly influenced the development of decision support systems, particularly through the integration of deep learning models. However, the black-box nature of these models has raised concerns regarding transparency, interpretability, and trust. To address these challenges, the concept of Explainable Artificial Intelligence (XAI) has emerged as a critical research area. XAI aims to make deep learning models more interpretable by providing insights into their decision-making processes, thereby enhancing reliability and user confidence in sensitive application domains.

One of the key contributions in this domain is presented by **Dr. Gokila Devi Mathialagan [1]**, who explores the application of explainable AI techniques in healthcare decision support systems. The study demonstrates that incorporating explainability into deep learning models enables accurate predictions while offering clear insights into the reasoning behind decisions. This approach is particularly important in clinical scenarios, where transparency is essential for gaining the trust of medical professionals. The findings emphasize that explainable models not only improve interpretability but also support ethical decision-making in high-risk environments.

Further advancements in interpretable deep learning are discussed by **T. Vengatesh [2]**, who focuses on enhancing transparency in decision-making systems. The study reviews various explainability techniques, including feature importance analysis and attention mechanisms, which help in understanding model behavior. The research concludes that these techniques significantly improve the interpretability of deep learning models without compromising their predictive performance. This work highlights the growing importance of transparency in modern AI systems and reinforces the need for explainable frameworks in decision support applications.

A comprehensive analysis of explainable AI in clinical environments is provided by **M. Salimparsa [3]**, who examines the limitations of deep learning models caused by their lack of transparency. The study identifies key gaps in existing research and emphasizes that the black-box nature of models can lead to reduced trust, as well as ethical and legal concerns. The author concludes that integrating explainability techniques into deep learning models is essential for improving accountability, transparency, and user confidence. These insights strongly align with the motivation behind developing robust and interpretable decision support systems.

In addition to these contributions, **T. Ahmad [4]** focuses on practical approaches to interpreting deep learning models using widely adopted techniques such as SHAP and LIME. The study demonstrates how these methods quantify the contribution of

individual input features to the final prediction, making the decision-making process more understandable. The results indicate that explainable AI enhances trust and ensures ethical compliance without significantly affecting model accuracy. This reinforces the importance of incorporating interpretability techniques into modern deep learning systems.

Moreover, **G.K. Kostopoulos [5]** provides a detailed review of explainable AI-based decision support systems across multiple domains. The study categorizes various explainability approaches and evaluates their effectiveness in addressing the limitations of traditional deep learning models. The findings highlight that achieving a balance between predictive performance and interpretability is crucial for building reliable systems. The increasing demand for transparent AI solutions further emphasizes the need for integrating explainability into decision support frameworks.

Overall, the existing literature clearly demonstrates that while deep learning models offer high predictive accuracy, their lack of interpretability poses significant challenges. Explainable AI techniques such as SHAP, LIME, attention mechanisms, and feature importance analysis play a vital role in overcoming these limitations. Despite notable progress, challenges such as computational complexity, scalability, and standardization of explanation methods remain. Addressing these challenges is essential for developing robust, transparent, and trustworthy decision support systems, which forms the primary objective of the proposed project.

3. SYSTEM ANALYSIS

3.1 Existing Systems

Existing decision support systems primarily rely on traditional machine learning models or complex deep learning architectures to automate decision-making. These systems are designed to analyze large datasets and provide predictions or recommendations based on learned patterns. In many real-world applications such as finance, healthcare, and risk assessment, deep learning models like neural networks are widely used due to their high accuracy and ability to handle complex relationships in data.

However, most existing systems focus mainly on performance metrics such as accuracy and efficiency, while giving little importance to transparency and interpretability. The decision-making process remains hidden from users, making it difficult to understand how input features influence the final outcome. As a result, these systems are often treated as black-box models.

Limitations of Existing System:

- Lack of transparency in decision-making processes.
- Difficulty in explaining model predictions to end users.
- Reduced trust and acceptance among stakeholders.
- Inability to identify bias or unfair decision patterns.
- Challenges in ethical compliance and regulatory requirements.
- Limited support for accountability in critical decision environments.

3.2 Proposed System

The proposed system aims to overcome the limitations of existing decision support systems by integrating explainable artificial intelligence techniques with deep learning models. This project develops an explainable deep learning–based decision support system that provides both accurate predictions and clear explanations for each decision.

The system uses a deep neural network to analyze input data and generate predictions. To ensure transparency, explainability techniques such as SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations) are applied to identify the contribution of each input feature to the final decision. This enables users to understand the reasoning behind model outputs, thereby improving trust and ethical compliance.

Model Workflow:

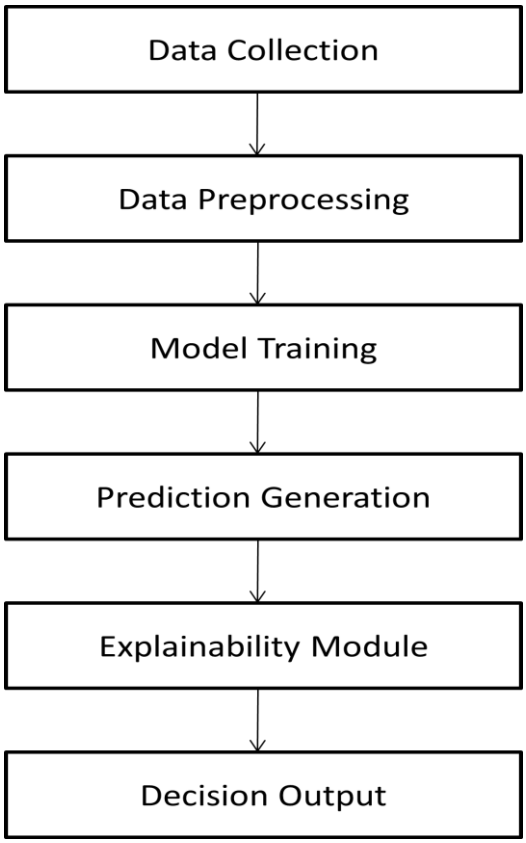

Figure 3.2.1 Model Workflow

The workflow of the proposed system consists of the following steps:

1. **Data Collection:** Input data relevant to decision-making is collected from structured datasets.
2. **Data Preprocessing:** The collected data is cleaned, normalized, and transformed to improve model performance.
3. **Model Training:** A deep learning model is trained using preprocessed data to learn decision patterns.
4. **Prediction Generation:** The trained model generates predictions based on new input data.
5. **Explainability Module:** Feature importance analysis is applied to explain the model's predictions.
6. **Decision Output:** The system provides both the prediction and its explanation to the user.

Advantages of Proposed System:

The proposed system offers several advantages over existing systems:

- Provides transparent and interpretable decision outputs.
- Enhances user trust and confidence in the system.
- Supports ethical and accountable decision-making.
- Helps identify influential features in the decision process.
- Maintains high prediction accuracy while offering explainability.
- Suitable for deployment in critical and real-world decision support applications.

4. REQUIREMENT ANALYSIS AND SPECIFICATIONS

4.1 Hardware Requirements

- CPU
  - Intel Core i5 (8^{th} gen or above)
  - AMD Ryzen (3000 series or above)
- GPU
  - **Minimum :** NVIDIA GTX 1050 / GTX 1650 (4 GB VRAM)
  - **Recommended :** NVIDIA GTX 1660 / RTX 2060 or higher (6–8 GB VRAM)
  - **CUDA Support :** CUDA-enabled NVIDIA GPU
- Minimum RAM and storage requirements
  - RAM
    - Minimum : 8GB
    - Recommended : 16GB
  - Storage
    - Minimum : 10GB – 20GB free disk space
    - Storage type : SSD

4.2 Software Requirements

- Operating System
  - Windows 10 or above
  - Linux
- Python programming language
- Deep learning frameworks
  - TensorFlow 2.8+ with Keras API
  - PyTorch 1.0 or above
- Libraries and Tools
  - NumPy, Pandas
  - Scikit-learn
  - TensorFlow / PyTorch
  - SHAP, LIME
  - Matplotlib, Seaborn

4.3 Functional Requirements

- Data input and validation.
- Model training and validation.
- Prediction generation.
- Explanation generation.
- Visualization of results and explanations.

5. DESIGN ARCHITECTURE

The architecture of the proposed **Explainable Deep Learning Decision Support System** is designed to ensure accurate decision-making while maintaining transparency, interpretability, and ethical compliance. The system is modular in nature, where each component performs a specific function and collectively contributes to generating explainable decisions. The architecture flow begins with data input and ends with decision output along with explanations and audit support.

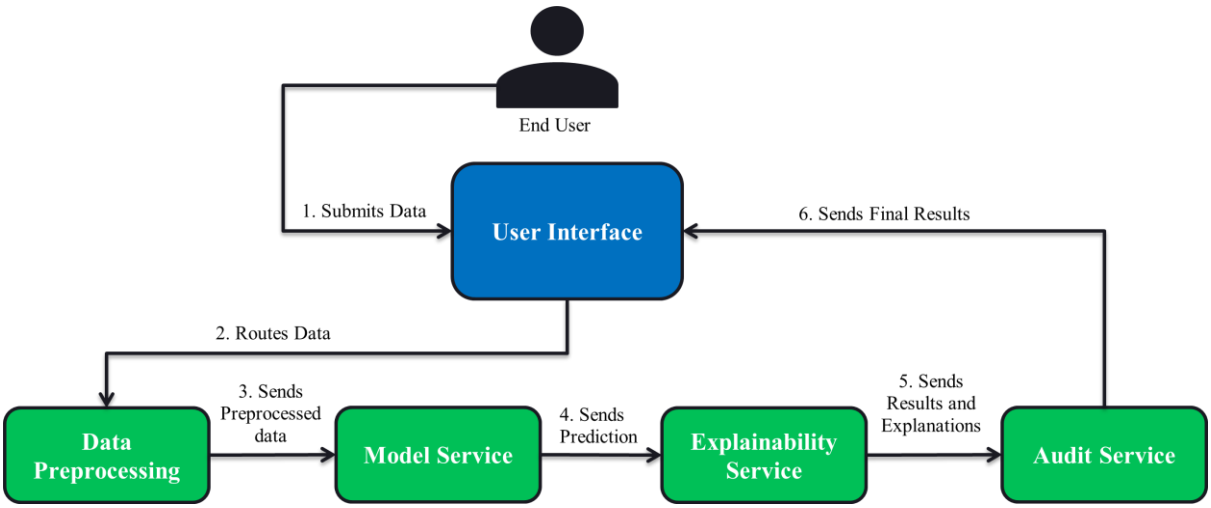

Figure 4.0.1 Design Architecture

5.1 Data Preprocessing Module:

The Data Management module is responsible for handling the input data required for the decision support system. It includes the following processes:

- **Data Collection:** Gathers structured input data relevant to the decision-making process.
- **Preprocessing:** Cleans the data by handling missing values, noise, and inconsistencies.
- **Feature Scaling:** Normalizes and standardizes features to improve model training efficiency.
- **Storage:** Stores processed data securely for training and prediction purposes

5.2 Model Service:

The core of the system is the Deep Learning Model, which uses architectures such as DNN or CNN or RNN depending on the nature of the data. This module performs two primary functions:

- **Training:** Learns complex patterns from processed input data using an optimizer and loss function.
- **Prediction:** Generates decision predictions for new or unseen data.

The optimizer and loss function play a crucial role in minimizing prediction error and improving model accuracy. Once the model is trained, it produces reliable predictions that are passed to both the decision output and explainability modules.

5.3 Explainability Service:

The Explainability Module addresses the black-box nature of deep learning models by providing insights into the decision-making process. This module includes:

- **Feature Importance Analysis:** Identifies which input features contribute most to the prediction.
- **SHAP and LIME:** Generates local and global explanations for individual predictions.
- **Attention Mechanism:** Highlights important data patterns influencing the model's decisions.

This module interacts bidirectionally with the deep learning model to interpret predictions and make them understandable to users.

5.4 Audit Service:

The Audit and Compliance module ensures ethical and responsible use of the system. It includes:

- **Fairness Check:** Detects potential bias in model decisions.
- **Performance Metrics:** Evaluates model accuracy, precision, and reliability.
- **Reporting:** Generates logs and reports for accountability and regulatory compliance.

Feedback from this module can be used to improve model performance and maintain ethical standards.

5.5 User Interface Module:

The User Interface module acts as the interaction layer between the system and the user. It presents:

- **Results Display:** Shows the final decision output generated by the model.
- **Visual Explanations:** Displays graphs, charts, and explanation summaries from the explainability module.

This module ensures that complex model outputs are presented in a user-friendly and interpretable format.

5.6 Decision Outputs and Explanation Outputs:

The system generates two types of outputs:

- **Decision Output:** Represents the final prediction or recommendation provided by the deep learning model.
- **Explanation Outputs:** Provide detailed reasoning behind the decision, including feature contributions and model behavior.

Both outputs are essential for improving transparency and user trust.

5.7 Overall System Flow:

The overall workflow begins with data input and preprocessing, followed by model training and prediction. Explainability techniques interpret the predictions, which are then presented to users through the interface. Finally, audit and compliance mechanisms ensure fairness, transparency, and accountability, completing the decision support lifecycle.

6. FEASIBILITY STUDY

The feasibility study evaluates the practicality and viability of the proposed Explainable Deep Learning Decision Support System from economic, technical, operational, and social perspectives. This analysis ensures that the system can be successfully developed, deployed, and used within real-world constraints.

6.1 Economic Feasibility:

The proposed system is economically feasible as it primarily relies on open-source technologies and software tools. Programming languages such as Python, along with libraries like TensorFlow, NumPy, Pandas, and Scikit-learn, are freely available and do not incur licensing costs. The use of standard computing resources reduces the need for expensive hardware or specialized infrastructure.

Additionally, the system does not require costly proprietary datasets, as it can operate on publicly available or synthetically generated data. Maintenance and operational costs are minimal, making the system suitable for academic and small-scale real-world deployment. Therefore, the overall development and execution cost of the proposed system is low and affordable.

6.2 Technical Feasibility:

The proposed system is technically feasible with existing technologies and tools. Deep learning frameworks and explainable AI techniques such as feature importance analysis, SHAP, and LIME are well-established and widely supported. The required hardware resources, such as a standard personal computer with sufficient memory and processing power, are adequate for training and testing the model.

Moreover, the system architecture is modular, making it easy to implement, test, and extend. The integration of explainability techniques with deep learning models is technically achievable and aligns with current research and industry practices. Hence, the system can be successfully developed using available technical expertise.

6.3 Operational Feasibility:

From an operational perspective, the proposed system is user-friendly and easy to operate. The user interface presents both decision outputs and explanations in a clear and understandable manner, requiring minimal technical knowledge from end users. This improves usability and acceptance among users.

The system can be integrated into existing decision-making workflows without significant disruption. Its explainable nature allows users to trust and validate the decisions, making it suitable for operational use in organizations. Additionally, routine maintenance and updates can be performed easily due to the use of well-documented technologies.

6.4 Social & Ethical Feasibility:

The proposed system is socially and ethically feasible as it emphasizes transparency, fairness, and accountability in decision-making. By providing explanations for model predictions, the system helps users understand and question automated decisions, reducing the risk of blind reliance on AI systems.

The inclusion of explainability and audit mechanisms supports ethical AI principles such as fairness, responsibility, and bias detection. This makes the system suitable for deployment in socially sensitive domains where ethical considerations are critical. Overall, the system promotes responsible and trustworthy use of artificial intelligence in decision support applications.

7. DATASET DESCRIPTION

The dataset used in this project is the **Loan Approval Prediction Dataset**, obtained from the Kaggle platform.

Dataset Structure:

Attribute	Description
loan_id	Unique identifier assigned to each loan application
no_of_dependents	Number of dependents supported by the applicant
Education	Education level of the applicant (Graduate / Not Graduate)
self_employed	Indicates whether the applicant is self-employed (Yes / No)
income_annum	Annual income of the applicant
loan_amount	Total loan amount requested
loan_term	Duration of the loan in years
cibil_score	Credit score representing the applicant's creditworthiness
residential_assets_value	Value of residential properties owned by the applicant
residential_assets_value	Value of commercial assets owned
luxury_assets_value	Value of luxury assets owned
bank_asset_value	Total value of bank assets held by the applicant
loan_status	Target variable indicating whether the loan is approved or rejected

Target Variable

The **loan_status** column serves as the **target variable** in this dataset.

It has two possible values

- **Approved** – The loan application is accepted by the financial institution.
- **Rejected** – The loan application is denied due to financial risk or insufficient eligibility

Features Used in this Project:

Although the dataset contains several attributes, this project focuses on a subset of the most influential financial factors to develop an interpretable model.

The selected features include:

- Annual Income
- CIBIL Score
- Loan Amount
- Loan Term

8. TOOLS, FRAMEWORKS, AND LIBRARIES

The successful implementation of the proposed Explainable Deep Learning Decision Support System relies on a combination of programming tools, deep learning frameworks, and supporting libraries. These technologies enable efficient data handling, model development, training, evaluation, visualization, and explainability.

Python:

Python serves as the primary programming language for the development of this project due to its simplicity, readability, and extensive ecosystem of scientific and machine learning libraries. Its high-level syntax enables rapid development and easy debugging, while its compatibility with deep learning frameworks makes it ideal for implementing explainable AI systems. Python supports both procedural and object-oriented programming paradigms, providing flexibility in system design.

- **Pandas:** Pandas is a high-level data analysis and manipulation library built on top of NumPy. It introduces useful data structures such as DataFrame and Series, which allow machine learning practitioners to clean, transform and analyze structured data efficiently before feeding it into models.
    - Used for data cleaning, transformation and preparation
    - Handles missing, inconsistent and categorical data
    - Simplifies exploratory data analysis
    - Integrates seamlessly with ML and visualization libraries

- **NumPy:** NumPy is a fundamental numerical computing library in Python that provides support for large, multi-dimensional arrays and matrices, along with a comprehensive collection of mathematical functions. In machine learning, NumPy is primarily used for handling numerical data, performing vectorized operations and implementing low-level mathematical computations efficiently.
    - Used for numerical feature representation and transformation
    - Enables fast mathematical operations through vectorization
    - Serves as the computational backbone for many ML libraries
    - Efficient memory management for large datasets

- **Scikit-learn:** Scikit-learn is used for preprocessing, dataset splitting, performance evaluation, and feature importance analysis. It provides utilities such as:
    - Train-test splitting
    - StandardScaler for feature normalization
    - Confusion matrix computation
    - Accuracy, precision, and recall metrics
    - Permutation feature importance

- **SHAP:** SHAP is an explainable AI library used to interpret model predictions. It is based on cooperative game theory and calculates Shapley values to measure the contribution of each feature toward a specific prediction. In this project, SHAP is used to generate local explanations for individual predictions, helping users understand how input features influence the decision outcome. This improves transparency and builds trust in the system.

- **LIME:** LIME refers to Local Interpretable Model-agnostic Explanations. The beauty of LIME its accessibility and simplicity. The core idea behind LIME though exhaustive is really intuitive and simple! Let's dive in and see what the name itself represents:
  Model agnosticism refers to the property of LIME using which it can give explanations for any given supervised learning model by treating it as a 'black box' separately. This means that LIME can handle almost any model that exists out there in the wild!
  Local explanations mean that LIME gives explanations that are locally faithful within the surroundings or vicinity of the observation/sample being explained.

- **Flask:** Flask is a lightweight Python web framework used to develop the user interface of the decision support system. It handles HTTP requests, user input submission, and rendering of prediction results. Flask enables seamless integration between the backend deep learning model and the frontend interface, allowing users to interact with the system through a web-based application.

- **TensorFlow:** TensorFlow is a useful open-source deep learning framework developed by Google. It is designed for building, training and deploying large-scale neural networks and supports both research and production-level machine learning systems.
  - Used for deep learning and neural networks
  - Supports GPU and distributed training
  - Highly scalable and production-ready
  - Flexible model architecture design

- **Keras:** Keras is a high-level neural network API that simplifies deep learning model development. It abstracts much of the complexity involved in building neural networks, making it especially suitable for beginners and rapid prototyping.
  - Simplifies neural network creation
  - Requires minimal code
  - Supports both regression and classification
  - Improves development speed

- **Matplotlib:** Matplotlib is a comprehensive data visualization library used to create static and interactive plots. In machine learning, it plays a critical role in understanding data distributions, detecting patterns and interpreting model performance through graphical representations.
    - Used for Visualizing datasets and model output
    - Helps identify trends, skewness and imbalances
    - Supports custom and publication-quality plots
    - Essential for result interpretation

- **Seaborn:** Seaborn is a statistical data visualization library built on Matplotlib. It is designed to create informative and visually appealing plots that help in understanding relationships between variables during exploratory data analysis.
    - Used for exploratory data analysis
    - Works directly with pandas DataFrames
    - Produces cleaner statistical plots
    - Enhances data interpretation

- **Model Logs and Saved Artifacts:** Model logs and saved artifacts include trained model files, preprocessing scalars, and evaluation outputs. These artifacts support reproducibility, performance analysis, and future model reuse, contributing to transparency and audit ability of the system.

9. ALGORITHMS

9.1 Back Propagation

Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs. It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases. Combined with optimization techniques like gradient descent, Backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.

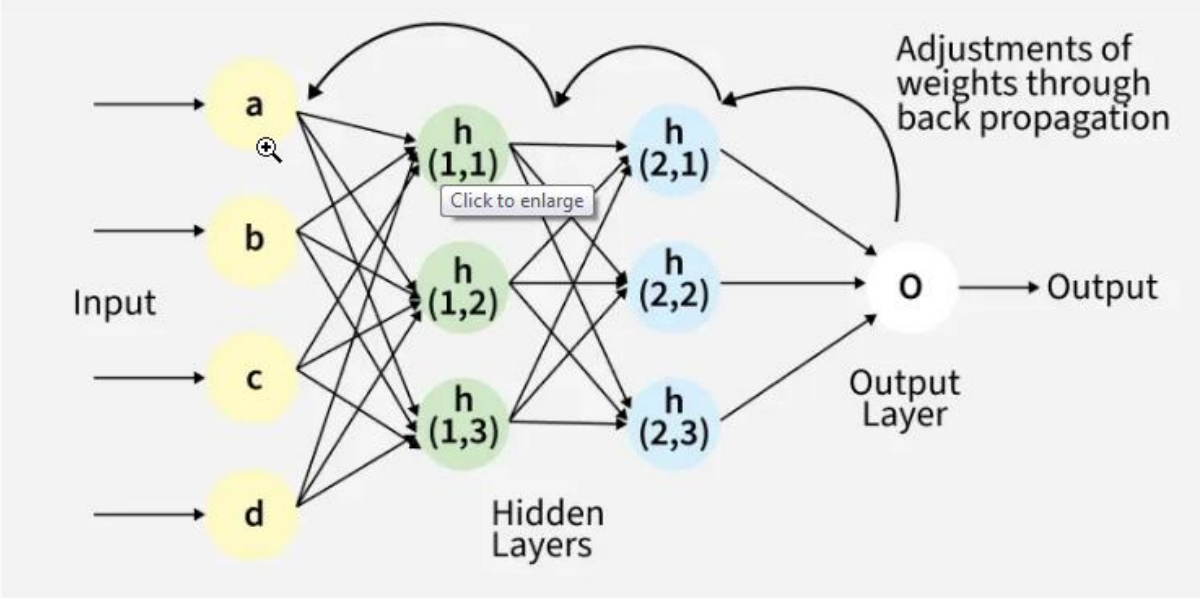

Figure 9.1.1 Back Propagation

Back Propagation plays a critical role in how neural networks improve over time. Here's why:

1. **Efficient Weight Update**: It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
2. **Scalability**: The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
3. **Automated Learning**: With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

Working of Back Propagation Algorithm:

1. **Forward Pass Work**

   In forward pass the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a

network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2. Before applying an activation function, a bias is added to the weighted inputs.

Each hidden layer computes the weighted sum (`a`) of the inputs then applies an activation function like ReLU (Rectified Linear Unit) to obtain the output (`o`). The output is passed to the next layer where an activation function such as softmax converts the weighted outputs into probabilities for classification.

2. Backward Pass Work

In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the Mean Squared Error (MSE) given by:

MSE=(Predicted Output−Actual Output)^2

Once the error is calculated the network adjusts weights using gradients which are computed with the chain rule. These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration. The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.

3. ReLU Activation Function

Rectified Linear Unit (ReLU) is a popular activation functions used in neural networks, especially in deep learning models. It has become the default choice in many architectures due to its simplicity and efficiency. The ReLU function is a piecewise linear function that outputs the input directly if it is positive; otherwise, it outputs zero.

In simpler terms, ReLU allows positive values to pass through unchanged while setting all negative values to zero. This helps the neural network maintain the necessary complexity to learn patterns while avoiding some of the pitfalls associated with other activation functions, like the vanishing gradient problem.
The ReLU function can be described mathematically as follows:

$$f(x)=\max(0,x)$$

Where:

- x is the input to the neuron.
- The function returns x if x is greater than 0.
- If x is less than or equal to 0, the function returns 0.

The formula can also be written as:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

This simplicity is what makes ReLU so effective in training deep neural networks, as it helps to maintain non-linearity without complicated transformations, allowing models to learn more efficiently.

4. Softmax Activation Function

In Deep Learning, activation functions are important because they introduce non-linearity into neural networks allowing them to learn complex patterns. Softmax Activation Function transforms a vector of numbers into a probability distribution, where each value represents the likelihood of a particular class. It is especially important for multi-class classification problems.

- Each output value lies between 0 and 1.
- The sum of all output values equals 1.

This property makes Softmax ideal for scenarios where each output neuron represents the probability of a distinct class.

Softmax Function

For a given vector, $z = [z_1, z_2, \ldots, z_n]$ the Softmax function is defined as:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

where:

- e^{z_j}: Exponentiation of the input value.
- $\sum_{j=1}^{n} e^{z_j}$: Sum of all exponentiated values to normalize outputs.

Key Characteristics:

- **Normalization:** Converts logits into a probability distribution where the sum equals 1.
- **Exponentiation:** Amplifies larger values making the model's confidence more pronounced.
- **Differentiable:** Enables gradient-based optimization during Backpropagation.
- **Probabilistic Interpretation:** Makes output easier to interpret as class likelihoods.

Working of Softmax Function:

Softmax converts a vector of raw scores into a probability distribution.

- **Input Scores:** Take the raw output vector from the model. These values can be any real numbers.
- **Exponentiate:** Apply *e^x* to make every value positive and amplify differences.
- **Sum of exponentials:** Compute the normalising constant *Z*=∑*e^x′*
- **Normalize:** Divide each exponent by Z to get probabilities *pi*=*e^x′**i / Z.
- **Output (Probabilities):** Final probability vector can be used with argmax to pick the predicted class.

9.2 Adam Optimizer

The Adam optimizer is used to enhance the training performance of the deep neural network by combining the advantages of gradient descent and momentum, making it suitable for complex deep learning models used in explainable decision support systems.

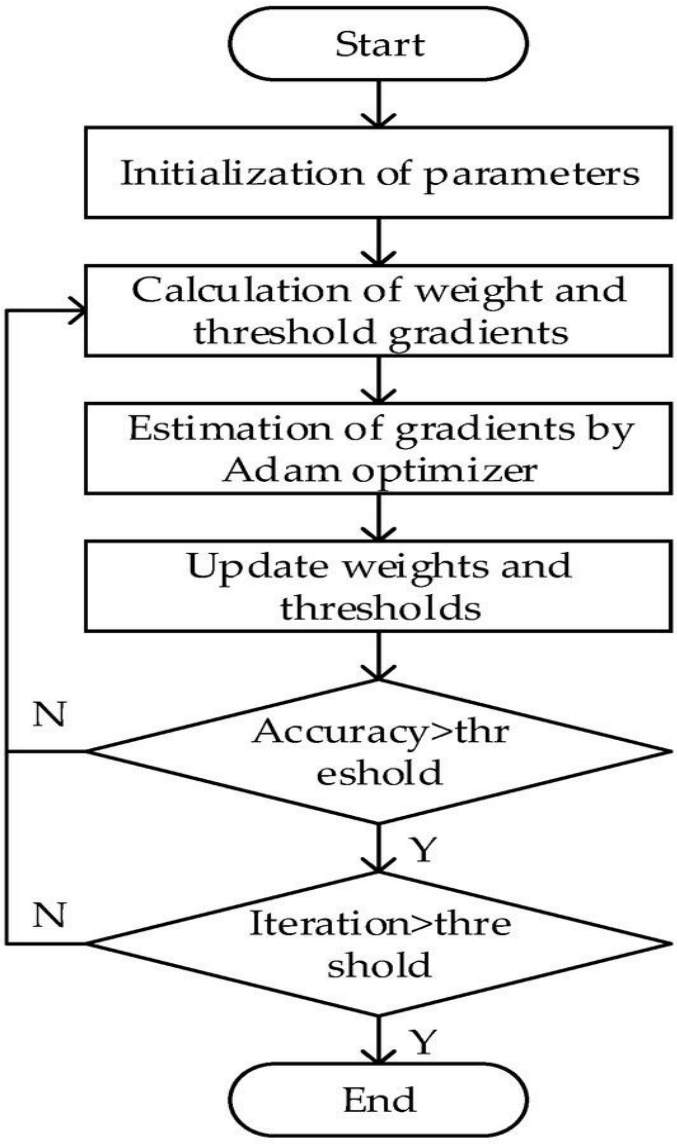

Figure 9.2.1 Adam Optimizer

Working of Adam Optimizer Algorithm:

- **Input:**
  Training dataset, learning rate α, momentum parameters β_1 and β_2
- **Steps:**
  1. Initialize network parameters (weights and biases) and set first-moment vector m0=0and second-moment vector v0=0.

2. For each training iteration *t*, compute the gradient of the loss function with respect to the model parameters.
3. Update the first-moment estimate (mean of gradients) using:

$$m_t=\beta_1 m_{t-1}+(1-\beta_1)g_t$$

4. Update the second-moment estimate (uncentered variance of gradients) using:

$$v_t=\beta_2 v_{t-1}+(1-\beta_2)g_t^2$$

5. Apply bias correction to obtain unbiased estimates of the first and second moments.
6. Update the model parameters using the corrected moment estimates and learning rate:

$$\theta_{t+1}=\theta_t-(\alpha\cdot m_t\ /\ \sqrt{v_t}+\varepsilon)$$

7. Repeat steps 2 to 6 for all training samples and epochs until convergence is achieved.

- **Output:**
  Optimized neural network parameters with faster and stable convergence

9.3 SHAP

The SHAP (SHapely Additive exPlanations) algorithm explains model predictions by computing the contribution of each input feature using Shapley values derived from cooperative game theory. It fairly distributes the prediction output among input features, allowing users to understand how each feature influences the final prediction.

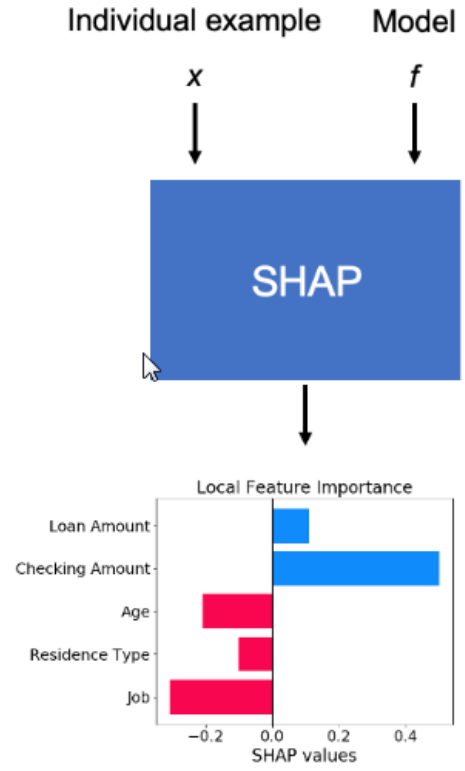

Figure 9.3.1 Example for working of SHAP

Steps:

1. **Train the model**
   Train the deep learning model using the prepared dataset to learn the relationship between input features and target output.
2. **Select an instance**
   Choose a specific data instance xxx from the dataset whose prediction needs to be explained.
3. **Generate feature subsets**
   Create different subsets of input features by including or excluding specific features from the model input.
4. **Compute model predictions**
   For each subset of features, evaluate the model prediction to observe how the output changes when certain features are present or absent.
5. **Calculate marginal contribution**
   Measure the change in prediction caused by adding a feature to a subset of other features. This represents the marginal contribution of that feature.
6. **Compute Shapley values**
   Calculate the average marginal contribution of each feature across all possible feature combinations.
7. **Assign feature importance**
   The computed SHAP values represent the contribution of each feature toward increasing or decreasing the model's prediction.
8. **Visualize explanations**
   Present the SHAP values using visualization techniques such as feature importance plots, summary plots, or force plots to help users interpret the model's decisions.

Mathematical Formulation:

The contribution of each feature is calculated using the Shapley value formula:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

Where,

ϕi - SHAP value of feature *i*

F – Set of all features

S – Subset of features excluding feature *i*

f(S) – Model prediction using features in subset S

f(S∪{i}) – Prediction where feature *i* is added

9.4 LIME

LIME explains the prediction of a complex machine learning model by approximating it with a simpler interpretable model around a specific data instance. It generates synthetic samples near the instance and uses them to train a local surrogate model that provides feature importance for that prediction.

Unlike global explanation techniques, LIME focuses on local interpretability by explaining individual predictions of a machine learning model.

Steps:

1. **Train the model**
   Train the machine learning or deep learning model using the dataset.
2. **Select an instance**
   Choose a specific data instance xxx whose prediction needs to be explained.
3. **Generate perturbed samples**
   Create multiple synthetic data points by slightly modifying the feature values of instance xxx.
4. **Obtain model predictions**
   Use the original model fff to predict outputs for all generated samples.
5. **Compute proximity weights**
   Assign higher weights to samples that are closer to the original instance xxx.
6. **Train an interpretable model**
   Fit a simple interpretable model (such as a linear regression model) using the weighted samples.
7. **Extract feature importance**
   Analyze the coefficients of the interpretable model to determine how each feature contributed to the prediction.
8. **Generate explanation**
   Present the most influential features as the explanation for the prediction.

Mathematical Formulation:

LIME explains a model by approximating it with a simpler interpretable model around a specific prediction. The objective is defined as:

$$\text{Explanation}(x) = \arg\min_{g \in G} \; L(f, g, \pi_x) + \Omega(g)$$

Where,

f – Original complex machine learning model

g – Interpretable model (e.g., linear model)

G – Set of possible interpretable models

$L(f,g,\pi_x)$ - Loss function measuring how well g approximates f

π_x - Local proximity measure around instance x

$\Omega(g)$ - Complexity penalty for the interpretable model

10. IMPLEMENTATION

10.1 Data Management Module

The data management module is responsible for collecting, preparing, and structuring the dataset used in the proposed explainable deep learning decision support system. The dataset employed in this project is a tabular dataset, which may be synthetic or obtained from real-world sources, consisting of multiple numerical and categorical features along with corresponding class labels. Initially, data cleaning methods are applied to ensure data quality by removing duplicate entries, correcting inconsistent values, and eliminating irrelevant or redundant attributes. These steps help reduce noise in the dataset and improve the reliability of the learning process.

Handling missing values is a critical step in the preprocessing phase, as incomplete data can negatively affect model performance. Missing values are addressed using appropriate statistical imputation techniques, such as replacing missing entries with mean or median values for numerical features. After handling missing data, feature scaling and normalization are applied to ensure that all input features are on a comparable scale. Z-score normalization is used to standardize feature values by transforming them to have zero mean and unit variance. This normalization improves the convergence speed of the deep learning model and prevents features with larger magnitudes from dominating the learning process. The entire data preprocessing pipeline is implemented using Python libraries such as Pandas for data manipulation, NumPy for numerical operations, and Scikit-learn for preprocessing and scaling.

10.2 Deep Learning Model

The deep learning model forms the core computational component of the proposed system and is designed to learn complex patterns from the preprocessed dataset. A Deep Neural Network (DNN) is implemented to perform supervised learning, where the model is trained using labeled data to predict output classes. The problem is formulated as a binary classification task, enabling the system to generate clear and interpretable decision outcomes. The DNN architecture consists of an input layer corresponding to the dataset features, multiple hidden layers for learning hierarchical representations, and an output layer that produces classification results.

During training, the model uses backpropagation to compute the gradients of the loss function with respect to network weights. These gradients are used to iteratively update the weights and biases in a direction that minimizes prediction error. Optimization of the learning process is achieved using the Adam optimizer, which combines the advantages of momentum and adaptive learning rates to ensure faster convergence and stable training. The training process is repeated over multiple epochs until the model achieves satisfactory performance. The model is developed and trained using TensorFlow and Keras frameworks, which provide high-level abstractions for building and training deep learning models efficiently. In addition to the implemented DNN, a conceptual study of Convolutional Neural Networks (CNNs) and Recurrent

Neural Networks (RNNs) is included to analyze their suitability for image-based and sequential data, respectively, and to identify possible future enhancements of the system.

10.3 Explainability Module

During training, the model uses backpropagation to compute the gradients of the loss function with respect to network weights. These gradients are used to iteratively update the weights and biases in a direction that minimizes prediction error. Optimization of the learning process is achieved using the Adam optimizer, which combines the advantages of momentum and adaptive learning rates to ensure faster convergence and stable training. The training process is repeated over multiple epochs until the model achieves satisfactory performance. The model is developed and trained using TensorFlow and Keras frameworks, which provide high-level abstractions for building and training deep learning models efficiently. In addition to the implemented DNN, a conceptual study of Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) is included to analyze their suitability for image-based and sequential data, respectively, and to identify possible future enhancements of the system.

In addition to permutation-based methods, the system incorporates documented explainability techniques such as SHAP and LIME to conceptually support both global and local interpretability. These techniques explain individual predictions by approximating the contribution of each feature to the model's output. Feature contribution analysis is used to interpret importance scores and provide meaningful insights into model behavior. The explainability module is implemented using Scikit-learn and NumPy, with documented references to SHAP and LIME libraries, ensuring that the decision support system remains transparent, interpretable, and trustworthy.

10.4 User Interface

The user interface provides a visualization layer that allows users to interact with the explainable decision support system. The predictions generated by the deep learning model are displayed along with visual explanations such as feature importance plots and confusion matrices. Visualization tools such as Matplotlib and Seaborn are used to present results in an interpretable format. This interface enables users to understand model predictions and improves transparency and decision confidence.

10.5 Audit and Compliance Module

Graphical representations such as feature importance plots and confusion matrices are generated to evaluate and communicate model performance. These visualizations help users analyze prediction accuracy, misclassification patterns, and overall system effectiveness. Visualization libraries such as Matplotlib and Seaborn are used to generate informative charts and plots. The user interface acts as a bridge between complex backend computations and end-user understanding, enhancing usability and decision confidence.

Performance metrics such as accuracy, precision, recall, and confusion matrices are used to evaluate system reliability and consistency. Model logs and saved artifacts are maintained to support reproducibility, traceability, and auditing of results. The system supports a human-in-the-loop decision-making approach, allowing users to review model predictions and explanations before taking final actions. By integrating ethical principles such as transparency, accountability, and user involvement, the audit and compliance module strengthens trust in the system and ensures responsible deployment of the explainable deep learning decision support system.

11. SYSTEM TESTING

System testing is an essential phase in the development of the proposed explainable deep learning decision support system. It ensures that all system modules function correctly both individually and collectively, and that the system meets the specified functional and non-functional requirements. The testing process validates the accuracy, reliability, interpretability, and ethical compliance of the system before deployment.

11.1 Testing Strategy:

The testing strategy adopted for this project includes both **unit testing** and **integration testing** to ensure correctness at different levels of system development. Unit testing is performed on individual modules such as data preprocessing, model training, prediction generation, and explainability components to verify that each module functions as intended in isolation. This helps in early identification and correction of errors at the module level.

Integration testing is then conducted to verify the interaction and data flow between different modules. The integration of data management with the deep learning model, explainability module, and user interface is tested to ensure seamless communication and correct output generation. This strategy ensures that the complete system works cohesively as a unified decision support framework.

11.2 Test Cases

Test cases are designed to validate both **functional behavior** and **performance aspects** of the system. Functional test cases focus on verifying core operations such as correct data input handling, successful model training, accurate prediction generation, and proper explanation output. These tests ensure that each system function operates according to the specified requirements.

Performance test cases are conducted to evaluate system efficiency, stability, and scalability. These tests assess the system's ability to process datasets of varying sizes, maintain consistent prediction accuracy, and generate explanations within acceptable time limits. Performance testing ensures that the system remains responsive and reliable under practical usage conditions.

Functional Test Cases:

1. **Data Input Test Case**
   This test case was implemented by providing different tabular datasets to the system, including valid datasets and datasets containing invalid formats or missing columns. The system was observed to verify whether it correctly accepts valid inputs and flags incorrect or incomplete data. This ensured proper validation of dataset structure before further processing.

2. **Data Preprocessing Test Case**
   This test case was implemented by applying data cleaning, missing value handling, and feature scaling methods to the input dataset. The processed output was examined to confirm that missing values were handled correctly, irrelevant entries were removed, and all feature values were normalized. Successful preprocessing confirmed readiness of the data for model training.
3. **Model Training Test Case**
   This test case was implemented by initiating the training process using the preprocessed dataset and monitoring the learning behavior of the deep neural network. The training loss and accuracy were observed across epochs to verify stable convergence without runtime errors. Successful completion confirmed correct implementation of the training process.
4. **Prediction Generation Test Case**
   This test case was implemented by passing unseen test data to the trained model and validating the generated binary classification outputs. The predicted labels were compared against expected outcomes to ensure consistency and correctness of predictions.
5. **Explainability Output Test Case**
   This test case was implemented by applying permutation feature importance and feature contribution analysis to trained model outputs. Feature importance scores and explanations were examined to ensure that influential features caused measurable changes in model performance and that explanation outputs aligned with prediction behavior.
6. **User Interface Display Test Case**
   This test case was implemented by verifying the visual outputs generated by the system, including prediction results, feature importance plots, and confusion matrices. The interface was tested to ensure all outputs were displayed clearly and accurately without distortion or misrepresentation.

Performance Test Cases:

1. **Model Accuracy Test Case**
   This test case was implemented by evaluating the trained model on a separate test dataset and computing the accuracy metric. The obtained accuracy value was analyzed to confirm acceptable classification performance.
2. **Precision and Recall Test Case**
   This test case was implemented by calculating precision and recall using confusion matrix values. These metrics were used to assess the correctness of positive predictions and the model's ability to identify relevant instances.
3. **Explainability Consistency Test Case**
   This test case was implemented by running the explainability module multiple times on the same input data and observing consistency in feature importance rankings. Stable explanation outputs confirmed reliability of the explainability methods.

4. **Response Time Test Case**

This test case was implemented by measuring the time taken by the system to generate predictions and explanations after input submission. The response time was evaluated to ensure timely system performance under normal operating conditions

11.3 Performance Metrics

Performance metrics are used to evaluate the effectiveness, reliability, and interpretability of the proposed explainable deep learning decision support system. These metrics help assess both the predictive performance of the model and the quality of explanations generated by the system.

1. **Confusion Matrix**

The confusion matrix is the easiest way to measure the performance of a classification problem where the output can be of two or more type of classes. A confusion matrix is nothing but a table with two dimensions viz. "Actual" and "Predicted" and furthermore, both the dimensions have "True Positives (TP)", "True Negatives (TN)", "False Positives (FP)", "False Negatives (FN)" as shown below –

Table 12.3.1 Confusion Matrix

		Actual	
		1	0
Predicted	1	True Positive(TP)	False Positive(FP)
	0	False Negative(FN)	True Negative(TN)

- **True Positives (TP):**
  It is the case where both the actual class and the predicted class of a data point are 1.
- **True Negatives (TN):**
  It is the case where both the actual class and the predicted class of a data point are 0.
- **False Positives (FP):**
  It is the case where the actual class of a data point is 0, but the predicted class of the data point is 1.
- **False Negatives (FN):**
  It is the case where the actual class of a data point is 1, but the predicted class of the data point is 0

2. **Accuracy**

Accuracy measures the overall correctness of the classification model. It represents the ratio of correctly predicted instances to the total number of instances evaluated. Accuracy is useful for understanding the general performance of the model when class distribution is balanced. A higher accuracy value indicates better classification capability of the deep learning model.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

3. Precision

Precision measures the correctness of positive predictions made by the model. It indicates how many of the instances predicted as positive are actually positive. Precision is particularly important in decision support systems where false positive predictions may lead to incorrect or undesired outcomes. A higher precision value reflects greater reliability of positive predictions.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

4. Recall

Recall measures the model's ability to correctly identify all relevant positive instances. It represents the proportion of actual positive cases that are correctly classified by the model. Recall is critical in applications where missing a positive instance can have serious consequences. A higher recall value indicates better sensitivity of the model.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

5. Explainability Quality

Explainability quality evaluates how effectively the system explains the predictions made by the deep learning model. This metric is assessed by analyzing the clarity, consistency, and relevance of feature importance and contribution outputs. High explainability quality indicates that the explanations are stable across multiple runs, align with model behavior, and are easily understandable by users. This metric ensures transparency, builds trust, and supports ethical decision-making in the system.

11.4 Model Evaluation and Testing Results

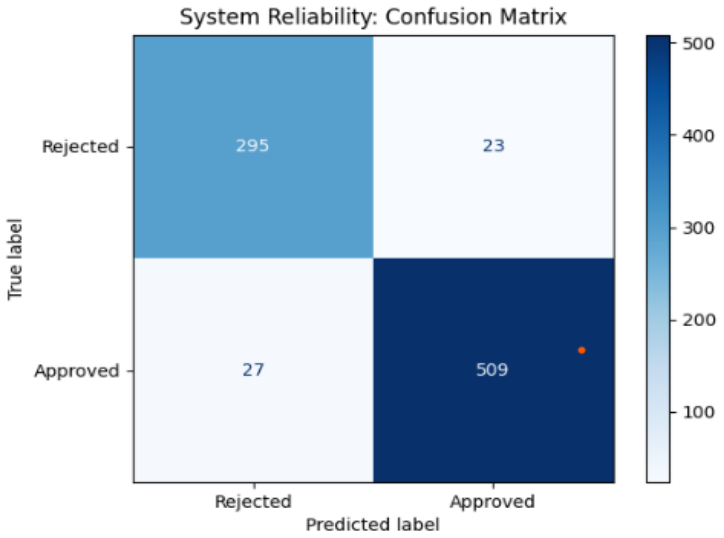

Figure 11.4.1 Confusion Matrix of Loan Approval Prediction Model Demonstrating Classification Outcomes

```
(venv) PS C:\Users\menih\OneDrive\Desktop\loan_app> python train_model.py
Starting Training...
Epoch 0 | Loss: 0.7486
Epoch 20 | Loss: 0.3563
Epoch 40 | Loss: 0.2298
Epoch 60 | Loss: 0.2007
Epoch 80 | Loss: 0.1916

==================================
  SYSTEM PERFORMANCE METRICS
==================================
Accuracy:  0.9239
Precision: 0.9385
Recall:    0.9403
F1-Score:  0.9394
==================================
Training Complete. Files saved successfully.
Performance Metrics Chart saved as performance_metrics.png
```

Figure 11.4.2 Model Training and Performance Metrics

Figure 11.4.3 User interface after model prediction 1

Figure 11.4.4 User interface after model prediction 2

12. CODE

12.1 Application Layer(Flask Web Interface) – app.py

This module handles:

- User input collection
- Model inference
- Explainability generation (SHAP + LIME)
- Visualization of explanations

```
import io, base64, torch, joblib, shap, lime, lime.lime_tabular
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from flask import Flask, render_template, request
from train_model import ExplainableDSSModel

app = Flask(__name__)
plt.switch_backend('Agg')

# ------------------------------
# Load Saved Model Components
# ------------------------------
scaler = joblib.load("scaler.pkl")
background = joblib.load("background.pkl")
training_data = joblib.load("training_data.pkl")

model = ExplainableDSSModel(4)
model.load_state_dict(torch.load("model.pth"))
model.eval()

# ------------------------------
# Prediction Function
# ------------------------------
def model_predict_probs(data):
    data_tensor = torch.tensor(data, dtype=torch.float32)

    with torch.no_grad():
        p = model(data_tensor).numpy()

    # Returns probability of rejection and approval
```

```
    return np.hstack((1-p, p))


# ------------------------------
# Global Explainability Objects
# ------------------------------
shap_explainer = shap.KernelExplainer(
    lambda x: model_predict_probs(x)[:, 1], background
)

lime_explainer = lime.lime_tabular.LimeTabularExplainer(
    training_data,
    feature_names=["Income", "CIBIL Score", "Loan Amount", "Loan Term"],
    class_names=['Rejected', 'Approved'],
    mode='classification'
)


# ------------------------------
# Home Route
# ------------------------------
@app.route('/')
def home():
    return render_template('index.html')


# ------------------------------
# Prediction Route
# ------------------------------
@app.route('/predict', methods=['POST'])
def predict():

    try:

        f = request.form

        user_inputs = {
            'income': f['income'],
            'cibil': f['cibil'],
            'loan': f['loan'],
            'term': f['term']
        }
```

```
raw = np.array([[
    float(f['income']),
    float(f['cibil']),
    float(f['loan']),
    float(f['term'])
]])

scaled = scaler.transform(raw)

# Model prediction
prob = model_predict_probs(scaled)[0][1]
result = "Approved" if prob > 0.5 else "Rejected"


# ------------------------------
# Explainability
# ------------------------------
shap_vals = shap_explainer.shap_values(
    scaled,
    nsamples=100
).flatten()

lime_exp = lime_explainer.explain_instance(
    scaled[0],
    model_predict_probs,
    num_features=4
)

lime_vals = [val for _, val in lime_exp.as_list()]


# ------------------------------
# Human Readable Explanation
# ------------------------------
features = [
    "Annual Income",
    "CIBIL Score",
    "Loan Amount",
    "Loan Term"
]

top_pos_idx = np.argmax(shap_vals)
top_neg_idx = np.argmin(shap_vals)
```

```
t_pos = features[top_pos_idx]
t_neg = features[top_neg_idx]

if result == "Approved":

    explanation = (
        f"The system **Approved** this loan because your "
        f"**{t_pos}** is very strong, which outweighed the "
        f"risk from your {t_neg}."
    )

else:

    explanation = (
        f"The system **Rejected** this loan primarily due to "
        f"your **{t_neg}**. Even though your {t_pos} was a "
        f"positive factor, it was not enough to offset the risk."
    )


# ------------------------------
# Visualization
# ------------------------------
plt.clf()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

sns.barplot(
    x=shap_vals,
    y=features,
    ax=ax1,
    palette="RdBu"
)

ax1.set_title("SHAP (Global Consistency)")


sns.barplot(
    x=lime_vals,
    y=features,
    ax=ax2,
    palette="viridis"
```

```
            )

            ax2.set_title("LIME (Local Reason)")


            plt.tight_layout()

            buf = io.BytesIO()

            plt.savefig(
                buf,
                format='png',
                bbox_inches='tight'
            )

            plot_url = base64.b64encode(
                buf.getvalue()
            ).decode('utf-8')


            return render_template(
                'index.html',
                result=result,
                prob=f"{prob*100:.1f}%",
                plot=plot_url,
                explanation=explanation,
                user_inputs=user_inputs
            )

        except Exception as e:
            return f"Error during inference: {str(e)}"


    if __name__ == '__main__':
        app.run(debug=True)
```

12.2 Model Training Module – train_model.py

This module performs:

- Dataset preprocessing
- Neural network training
- Performance evaluation
- Model and scaler saving

```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# ------------------------------
# Neural Network Architecture
# ------------------------------
class ExplainableDSSModel(nn.Module):

    def __init__(self, input_dim):

        super(ExplainableDSSModel, self).__init__()

        self.net = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, 1),
            nn.Sigmoid()
        )
```

```
    def forward(self, x):

        return self.net(x)


# ------------------------------
# Model Training Function
# ------------------------------
def train():

    try:

        df = pd.read_csv("loan_approval_dataset.csv")

    except FileNotFoundError:

        print("Dataset not found.")
        return

    df.columns = df.columns.str.strip()

    y = (
        df['loan_status'].str.strip() == 'Approved'
    ).astype(float).values

    features = [
        'income_annum',
        'cibil_score',
        'loan_amount',
        'loan_term'
    ]

    X = df[features].values


    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


model = ExplainableDSSModel(len(features))

criterion = nn.BCELoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=0.01
)


print("Training Started")

for epoch in range(100):

    model.train()

    inputs = torch.tensor(
        X_train_scaled,
        dtype=torch.float32
    )

    labels = torch.tensor(
        y_train,
        dtype=torch.float32
    ).reshape(-1, 1)


    optimizer.zero_grad()

    outputs = model(inputs)

    loss = criterion(outputs, labels)

    loss.backward()

    optimizer.step()
```

```
    if epoch % 20 == 0:

        print(
            f"Epoch {epoch} | Loss: {loss.item():.4f}"
        )


# ------------------------------
# Evaluation
# ------------------------------
model.eval()

with torch.no_grad():

    test_inputs = torch.tensor(
        X_test_scaled,
        dtype=torch.float32
    )

    y_prob = model(test_inputs).numpy()

    y_pred = (y_prob > 0.5).astype(int)


acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)


print("\nModel Performance")

print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)


torch.save(model.state_dict(), "model.pth")

joblib.dump(scaler, "scaler.pkl")
```

```
    joblib.dump(
        X_train_scaled[:100],
        "background.pkl"
    )

    joblib.dump(
        X_train_scaled,
        "training_data.pkl"
    )


    cm = confusion_matrix(y_test, y_pred)

    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=['Rejected', 'Approved']
    )

    plt.figure(figsize=(6,6))

    disp.plot(cmap='Blues')

    plt.title("Confusion Matrix")

    plt.savefig("confusion_matrix.png")

    plt.close()


if __name__ == "__main__":
    train()
```

12.3 User Interface – index.html

This file implements the web interface for Explainable AI decision support.

```
<!DOCTYPE html>
<html>

<head>

<title>XAI Decision Support</title>

<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">

</head>

<body>

<div class="container py-5">

<h2 class="text-center">
Explainable AI for Loan Approval
</h2>

<form action="/predict" method="post">

<label>Annual Income</label>
<input type="number" name="income">

<label>CIBIL Score</label>
<input type="number" name="cibil">

<label>Loan Amount</label>
<input type="number" name="loan">

<label>Loan Term</label>
<input type="number" name="term">

<button type="submit">
Predict
</button>

</form>
```

```
</div>

</body>
</html>
```

13. OUTPUT SCREENS

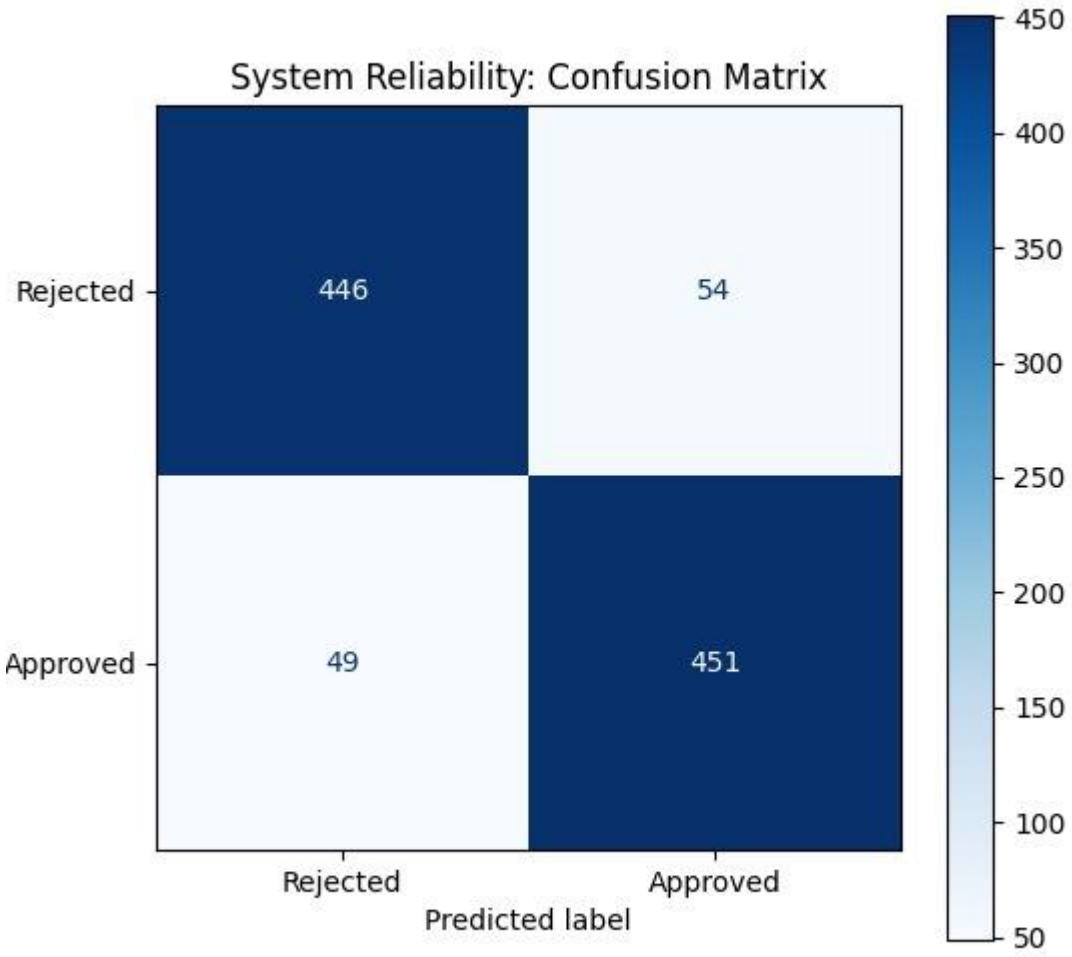

Figure13.1 Output Screen 1

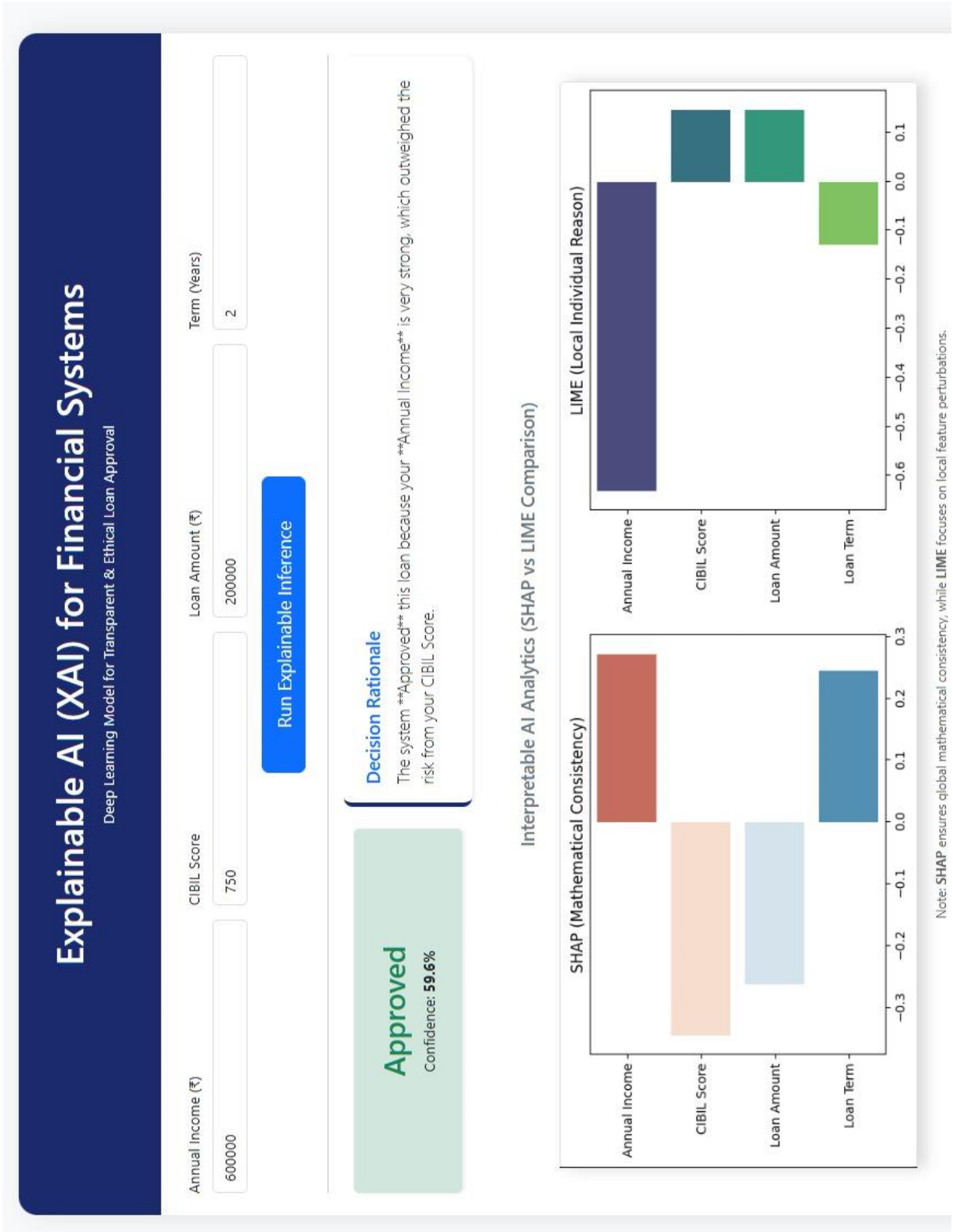

Figure 13.2 Output Screen 2

Figure13.3 Output Screen 3

14. CONCLUSION

This project successfully presents the design and implementation of an explainable deep learning–based decision support system aimed at improving transparency, interpretability, and ethical accountability in automated decision-making. By integrating a deep neural network with explainability techniques, the system addresses the limitations of traditional black-box models and enables users to understand not only *what* decisions are made but also *why* they are made.

The system effectively preprocesses tabular data through data cleaning, missing value handling, and feature normalization, ensuring reliable input for model training. A supervised deep learning model is trained to perform binary classification with satisfactory performance in terms of accuracy, precision, and recall. To enhance trust and transparency, SHAP and LIME are employed to identify influential features and explain model behavior. Visualization of predictions, feature importance, and performance metrics further improves interpretability and usability.

Additionally, the system incorporates audit and compliance considerations by supporting transparency analysis, bias awareness, and human-in-the-loop decision support. Comprehensive system testing validates the functional correctness, performance reliability, and explainability quality of the proposed solution. The results demonstrate that the system achieves a balanced combination of predictive accuracy and interpretability, making it suitable for ethical and transparent decision support applications.

In conclusion, this project highlights the importance of explainable artificial intelligence in modern decision support systems and demonstrates how deep learning models can be enhanced with interpretability mechanisms to build trustworthy, responsible, and user-centric AI solutions.

15. FUTURE ENHANCEMENT FOR EXPLAINABLE DEEP LEARNING MODEL FOR TRANSPARENT AND ETHICAL DECISION SUPPORT SYSTEMS

The proposed explainable deep learning decision support system provides a strong foundation for transparent and ethical AI-based decision-making; however, several enhancements can be explored in the future to improve its capability and applicability. One possible extension is the incorporation of more advanced explainability techniques such as integrated gradients, counterfactual explanations, and attention-based interpretability methods to provide deeper and more intuitive insights into model decisions.

The system can be further extended to support multi-class classification and regression problems, enabling its application across a wider range of real-world domains. Future work may also involve experimenting with more complex deep learning architectures such as Convolutional neural networks for image-based data and recurrent or transformer-based models for sequential and time-series data. This would enhance the system's adaptability to diverse data types.

Another important future direction is the integration of real-time decision support capabilities by deploying the system as a web-based or cloud-based application. This would allow scalable and continuous data processing, making the system suitable for enterprise-level usage. Additionally, automated bias detection and fairness evaluation mechanisms can be incorporated to strengthen ethical compliance and ensure unbiased decision-making.

Future enhancements may also focus on improving system performance through hyper parameter optimization, ensemble learning techniques, and automated model selection. Incorporating user feedback mechanisms within the human-in-the-loop framework can further improve model reliability and trust. Overall, the future scope of this project lies in expanding its scalability, interpretability, fairness, and real-world applicability, contributing to the development of responsible and trustworthy artificial intelligence systems.

16. REFERENCES

1. I. Goodfellow, Y. Bengio, and A. Courville Deep Learning. MIT Press, 2016. Available: https://www.deeplearningbook.org/

2. M. T. Ribeiro, S. Singh, and C. Guestrin "Why Should I Trust You?: Explaining the Predictions of Any Classifier," Proceeding of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016. Available: https://arxiv.org/abs/1602.04938

3. S. M. Lundberg and S.-I. Lee "A Unified Approach to Interpreting Model Predictions," Advances in Neural Information Processing System, 2017. Available: https://arxiv.org/abs/1705.07874

4. C. Molnar Interpretable Machine Learning: A Guide for Making Black Box Models Explainable, 2019. Available: https://christophm.github.io/interpretable-ml-book/

5. D. P. Kingma and J. Ba "Adam: A Method for Stochastic Optimization," International Conference on Learning Representations (ICLR), 2015. Available: https://arxiv.org/abs/1412.6980

6. F. Pedregosa et al "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available: https://scikit-learn.org/stable/

7. A. Paszke et al "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems (NeurIPS), 2019. Available: https://pytorch.org/

8. M. Abadi et al "TensorFlow: A System for Large-Scale Machine Learning," 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016. Available: https://www.tensorflow.org/

9. R. Guidotti et al "A Survey of Methods for Explaining Black Box Models," ACM Computing Surveys, 2018. Available: https://arxiv.org/abs/1802.01933

10. A. Adadi and M. Berrada "Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence," IEEE Access, vol. 6, pp. 52138–52160, 2018. Available: https://ieeexplore.ieee.org/document/8466590

11. A. Salih et al “A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME,” Advanced Intelligent Systems. Available:https://doaj.org/article/49fbd7132cef4df89d0ba1809822117c?utm_source=chatgpt.com

12. E. Amparore, A. Perotti, and P. Bajardi To Trust or Not to Trust an Explanation: Using LEAF to Evaluate Local Linear XAI Methods” Available: https://pubmed.ncbi.nlm.nih.gov/33977131/